Machine Learning
Semester: Winter 2025/2026
Group 45
Date: November 20, 2025

Vladimir Lazarev (459484)
Yura Min (461718)
Maxim Mjakinin (480592)
Julian Tanzil (382652)

Exercise Sheet 5

Exercise 1: Fisher Discriminant

a)

We consider Fisher's criterion

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top S_B \mathbf{w}}{\mathbf{w}^\top S_W \mathbf{w}}, \tag{1}$$

where

$$S_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top, \tag{2}$$

$$\mathbf{m}_k = \frac{1}{N_k} \sum_{n \in C_k} \mathbf{x}_n, \tag{3}$$

$$S_W = S_1 + S_2, \tag{4}$$

$$S_k = \sum_{n \in C_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^\top. \tag{5}$$

The numerator $\mathbf{w}^\top S_B \mathbf{w}$ is a quadratic function of $\mathbf{w}$. If we tried to maximize only $\mathbf{w}^\top S_B \mathbf{w}$ without any constraint, the solution would be trivial: we could increase $\|\mathbf{w}\|$ arbitrarily and the value would go to infinity.
Therefore we introduce a constraint on $\mathbf{w}$. The ratio $J(\mathbf{w})$ is invariant under rescaling of $\mathbf{w}$: for any scalar $\alpha \neq 0$,

$$J(\alpha\mathbf{w}) = \frac{(\alpha\mathbf{w})^\top S_B (\alpha\mathbf{w})}{(\alpha\mathbf{w})^\top S_W (\alpha\mathbf{w})} = \frac{\alpha^2 \mathbf{w}^\top S_B \mathbf{w}}{\alpha^2 \mathbf{w}^\top S_W \mathbf{w}} = J(\mathbf{w}). \tag{6}$$

So only the direction of $\mathbf{w}$ matters, not its length.
Because of this scale invariance we can fix the denominator to 1 and obtain an equivalent constrained problem:

$$\begin{aligned} \max_{\mathbf{w}} \quad & \frac{\mathbf{w}^\top S_B \mathbf{w}}{\mathbf{w}^\top S_W \mathbf{w}} \\ \text{s.t.} \quad & \mathbf{w}^\top S_W \mathbf{w} = 1 \end{aligned} \quad \Longleftrightarrow \quad \begin{aligned} \max_{\mathbf{w}} \quad & \mathbf{w}^\top S_B \mathbf{w} \\ \text{s.t.} \quad & \mathbf{w}^\top S_W \mathbf{w} = 1. \end{aligned} \tag{7}$$

Imposing $\mathbf{w}^\top S_W \mathbf{w} = 1$ fixes the scale of $\mathbf{w}$ (similar to constraining its norm) and removes the trivial solution, while leaving the maximizing direction unchanged.

Machine Learning
Semester: Winter 2025/2026
Group 45
Date: November 20, 2025

Vladimir Lazarev (459484)
Yura Min (461718)
Maxim Mjakinin (480592)
Julian Tanzil (382652)

b)

We consider the constrained maximization problem from part (a)

$$\max_{\mathbf{w}} \ \mathbf{w}^\top S_B \mathbf{w} \quad \text{s.t.} \quad \mathbf{w}^\top S_W \mathbf{w} = 1.$$

Introduce a Lagrange multiplier λ and define the Lagrangian

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top S_B \mathbf{w} - \lambda\big(\mathbf{w}^\top S_W \mathbf{w} - 1\big). \tag{8}$$

We now take the derivative with respect to $\mathbf{w}$ and set it to zero. Using the standard matrix–calculus rule

$$\frac{\partial}{\partial \mathbf{x}}\big(\mathbf{x}^\top B \mathbf{x}\big) = (B + B^\top)\mathbf{x},$$

we obtain

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (S_B + S_B^\top)\mathbf{w} - \lambda(S_W + S_W^\top)\mathbf{w} = \mathbf{0}. \tag{9}$$

Next we show that S_B and S_W are symmetric. For the between-class scatter,

$$\begin{aligned}
S_B^\top &= \big((\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top\big)^\top && \text{(use } (AB)^\top = B^\top A^\top\text{)} && (10)\\
&= (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top && \text{(use } (A^\top)^\top = A\text{)} && (11)\\
&= S_B. && && (12)
\end{aligned}$$

For the within-class scatter,

$$\begin{aligned}
S_W^\top &= (S_1 + S_2)^\top && \text{(use } (A+B)^\top = A^\top + B^\top\text{)} && (13)\\
&= S_1^\top + S_2^\top && && (14)\\
&= \sum_{n \in C_1} \big((\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^\top\big)^\top + \sum_{n \in C_2} \big((\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^\top\big)^\top && && (15)\\
&= \sum_{n \in C_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^\top + \sum_{n \in C_2} (\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^\top && && (16)\\
&= S_1 + S_2 = S_W, && && (17)
\end{aligned}$$

so S_W is also symmetric.
Therefore $S_B = S_B^\top$ and $S_W = S_W^\top$, and the stationarity condition simplifies to

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 2S_B\mathbf{w} - 2\lambda S_W \mathbf{w} = \mathbf{0}. \tag{18}$$

Dividing by 2 gives

$$S_B \mathbf{w} = \lambda S_W \mathbf{w}, \tag{19}$$

which is exactly the generalized eigenvalue problem stated in the exercise.

Machine Learning
Semester: Winter 2025/2026
Group 45
Date: November 20, 2025

Vladimir Lazarev (459484)
Yura Min (461718)
Maxim Mjakinin (480592)
Julian Tanzil (382652)

c)

From part (b) we know that any solution $\mathbf{w}$ of the optimization problem satisfies the generalized eigenvalue equation

$$S_B\mathbf{w} = \lambda S_W\mathbf{w}. \tag{20}$$

Assuming S_W is invertible, we left–multiply both sides by S_W^{-1}:

$$S_W^{-1}S_B\mathbf{w} = \lambda\mathbf{w}. \tag{21}$$

Using the definition

$$S_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top,$$

we obtain

$$S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top\mathbf{w} = \lambda\mathbf{w}. \tag{22}$$

The quantity $(\mathbf{m}_2 - \mathbf{m}_1)^\top\mathbf{w}$ is a scalar; denote

$$\alpha = (\mathbf{m}_2 - \mathbf{m}_1)^\top\mathbf{w} \in \mathbb{R}.$$

Then the previous equation can be written as

$$\alpha\, S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1) = \lambda\mathbf{w}, \tag{23}$$

or equivalently

$$S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1) = \frac{\lambda}{\alpha}\,\mathbf{w}. \tag{24}$$

Thus $\mathbf{w}$ is proportional to $S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$, i.e. there exists a nonzero scalar c such that

$$\mathbf{w}^* = c\, S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1). \tag{25}$$

Hence the solution of the optimization problem is equivalent (up to a scaling factor) to $S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$.

Exercise 2: Bounding the Error

a)

we basically have to show that:
$min\{P(w_1|x), P(w_2|x\} \leq \sqrt{P(w_1|x)P(w_2|x}$
so we rewrite it like this:
$min\{P(w_1|x), P(w_2|x\} \leq \sqrt{min\{P(w_1|x), P(w_2|x\} \cdot max\{P(w_1|x), P(w_2|x\}}$
<=>
$min\{P(w_1|x), P(w_2|x\} \cdot min\{P(w_1|x), P(w_2|x\} \leq min\{P(w_1|x), P(w_2|x\} \cdot max\{P(w_1|x), P(w_2|x\}$
this obviously holds as
$min\{P(w_1|x), P(w_2|x\} \leq max\{P(w_1|x), P(w_2|x\}$

b)

$P(\text{error}) = \int P(\text{error}|x)p(x)dx$. We use (a):

$$\begin{aligned} P(\text{error}) &\leq \int \sqrt{P(\omega_1|x)P(\omega_2|x)}p(x)dx \\ &= \int \sqrt{\frac{p(x|\omega_1)P(\omega_1)}{p(x)}\,\frac{p(x|\omega_2)P(\omega_2)}{p(x)}}\,p(x)dx \\ &= \sqrt{P(\omega_1)P(\omega_2)}\int \sqrt{p(x|\omega_1)p(x|\omega_2)}dx \end{aligned}$$

Substituting $\mathcal{N}(\mu, \Sigma)$ and $\mathcal{N}(-\mu, \Sigma)$:

$$\sqrt{p(x|\omega_1)p(x|\omega_2)} = k \cdot \exp\left(-\frac{1}{4}\left[(x-\mu)^\top\Sigma^{-1}(x-\mu) + (x+\mu)^\top\Sigma^{-1}(x+\mu)\right]\right)$$

Vladimir Lazarev (459484)
Yura Min (461718)
Maxim Mjakinin (480592)
Julian Tanzil (382652)

simplifying:

$$\begin{aligned}&(x-\mu)^\top\Sigma^{-1}(x-\mu)+(x+\mu)^\top\Sigma^{-1}(x+\mu)\\&=2x^\top\Sigma^{-1}x+2\mu^\top\Sigma^{-1}\mu\end{aligned}$$

thus:

$$\int\sqrt{p(x|\omega_1)p(x|\omega_2)}dx=\exp\left(-\frac{1}{2}\mu^\top\Sigma^{-1}\mu\right)\underbrace{\int k\cdot\exp\left(-\frac{1}{2}x^\top\Sigma^{-1}x\right)dx}_{=1}$$

$$P(\text{error})\leq\sqrt{P(\omega_1)P(\omega_2)}\exp\left(-\frac{1}{2}\mu^\top\Sigma^{-1}\mu\right)$$

Exercise 3: Fisher Discriminant

a)

from 1c we derived that the optimal Fisher discriminant $\mathbf{w}$ is proportional to:

$$\mathbf{w}\propto S_W^{-1}(\mathbf{m}_2-\mathbf{m}_1)$$

$$\mathbf{m}_2-\mathbf{m}_1=\begin{pmatrix}1\\1\end{pmatrix}-\begin{pmatrix}-1\\-1\end{pmatrix}=\begin{pmatrix}2\\2\end{pmatrix}$$

$$S_B=\begin{pmatrix}2\\2\end{pmatrix}\begin{pmatrix}2&2\end{pmatrix}=\begin{pmatrix}4&4\\4&4\end{pmatrix}$$

since $\Sigma_1=\Sigma_2=\Sigma$:

$$S_W=2\Sigma=2\begin{pmatrix}2&0\\0&1\end{pmatrix}=\begin{pmatrix}4&0\\0&2\end{pmatrix}$$

$$S_W^{-1}=\begin{pmatrix}1/4&0\\0&1/2\end{pmatrix}$$

Substitute into the equation for $\mathbf{w}$:

$$\mathbf{w}=\begin{pmatrix}1/4&0\\0&1/2\end{pmatrix}\begin{pmatrix}2\\2\end{pmatrix}=\begin{pmatrix}2/4\\2/2\end{pmatrix}=\begin{pmatrix}0.5\\1\end{pmatrix}$$

b)

from 1b we know :

$$S_B\mathbf{w}=\lambda S_W\mathbf{w}$$

we are looking for the smallest eigenvalue.

$$S_W^{-1}S_B\mathbf{w}=\lambda\mathbf{w}$$

$$S_B=\begin{pmatrix}4&4\\4&4\end{pmatrix}$$

$$A=\begin{pmatrix}1/4&0\\0&1/2\end{pmatrix}\begin{pmatrix}4&4\\4&4\end{pmatrix}=\begin{pmatrix}1&1\\2&2\end{pmatrix}$$

to find the eigenvalues: $\det(A-\lambda I)=0$

$$\det\begin{pmatrix}1-\lambda&1\\2&2-\lambda\end{pmatrix}=(1-\lambda)(2-\lambda)-2=0$$

$$2-3\lambda+\lambda^2-2=\lambda^2-3\lambda=\lambda(\lambda-3)=0$$

$\lambda_1=3$ (maximum) and $\lambda_2=0$ (minimum).
for the minimum ratio we use $\lambda=0$ and solve $A\mathbf{w}=0\mathbf{w}$:

$$\begin{pmatrix}1&1\\2&2\end{pmatrix}\begin{pmatrix}w_1\\w_2\end{pmatrix}=\begin{pmatrix}0\\0\end{pmatrix}$$

$w_1+w_2=0\implies w_1=-w_2$. A solution is:

$$\mathbf{w}=\begin{pmatrix}1\\-1\end{pmatrix}$$

sheet05-programming

November 19, 2025

1 Fisher Linear Discriminant

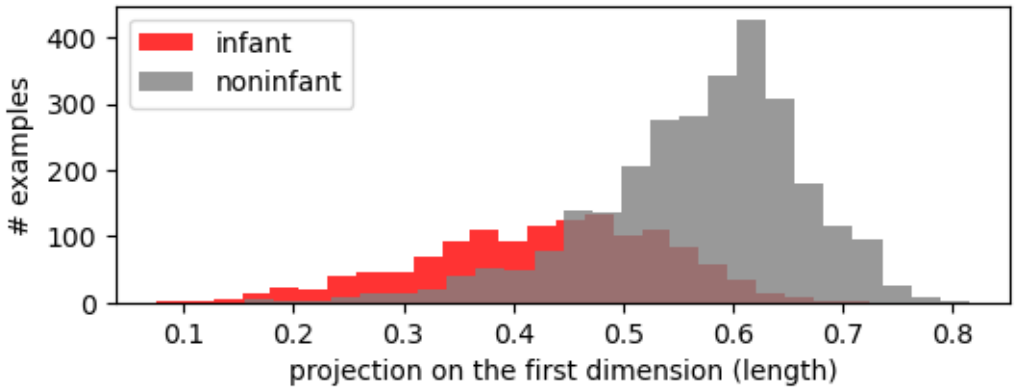
In this exercise, we apply Fisher Linear Discriminant as described in Chapter 3.8.2 of Duda et al. on the UCI Abalone dataset. A description of the dataset is given at the page https://archive.ics.uci.edu/ml/datasets/Abalone. The following two methods are provided for your convenience:

- **`utils.Abalone.__init__(self)`** reads the Abalone data and instantiates two data matrices corresponding to: *infant (I)*, *non-infant (N)*.
- **`utils.Abalone.plot(self,w)`** produces a histogram of the data when projected onto a vector `w`, and where each class is shown in a different color.

Sample code that makes use of these two methods is given below. It loads the data, looks at the shape of instantiated matrices, and plots the projection on the first dimension of the data representing the length of the abalone.

[11]:
```
%matplotlib inline
import utils,numpy
import numpy as np
import matplotlib.pyplot as plt

# Load the data
abalone = utils.Abalone()

# Print dataset size for each class
print(abalone.I.shape, abalone.N.shape)

# Project data on the first dimension
w1 = numpy.array([1,0,0,0,0,0,0])
abalone.plot(w1,'projection on the first dimension (length)')
```

```
(1342, 7) (2835, 7)
```



1.1 Implementation (10 + 5 + 5 = 20 P)

- **Create a function `w = fisher(X1,X2)` that takes as input the data for two classes and returns the Fisher linear discriminant.**
- **Create a function `objective(X1,X2,w)` that evaluates the objective defined in Equation 96 of Duda et al. for an arbitrary projection vector `w`.**
- **Create a function `z = phi(X)` that returns a quadratic expansion for each data point `x` in the dataset. Such expansion consists of the vector `x` itself, to which we concatenate the vector of all pairwise products between elements of `x`.** In other words, letting $x = (x_1, \ldots, x_d)$ denote the d-dimensional data point, the quadratic expansion for this data point is a $d \cdot (d+3)/2$ dimensional vector given by $\phi(x) = (x_i)_{1 \le i \le d} \cup (x_i x_j)_{1 \le i \le j \le d}$. For example, the quadratic expansion for $d = 2$ is $(x_1, x_2, x_1^2, x_2^2, x_1 x_2)$.

[12]:
```
def fisher(X1,X2):
    m1 = np.mean(X1, axis=0)
    m2 = np.mean(X2, axis=0)
    S1 = (X1 - m1).T @ (X1 - m1)
    S2 = (X2 - m2).T @ (X2 - m2)
    Sw = S1 + S2
    w = np.linalg.pinv(Sw) @ (m1 - m2)
    w /= np.linalg.norm(w)
    return w

def objective(X1,X2,w):
    m1 = np.mean(X1, axis=0)
    m2 = np.mean(X2, axis=0)
    S1 = (X1 - m1).T @ (X1 - m1)
    S2 = (X2 - m2).T @ (X2 - m2)
    Sw = S1 + S2

    dist = (m1 - m2).reshape(-1, 1)
    Sb = dist @ dist.T
```

```
    numerator = w.T @ Sb @ w
    denominator = w.T @ Sw @ w

    J = numerator / denominator
    #equation(96)=(103) if we read further below
    return J

def expand(X):
    n, d = X.shape
    m = d*(d+3)//2
    Z = np.zeros((n, m))

    i, j = np.triu_indices(d)

    for k in range(n):
        x = X[k]
        combinations = np.outer(x, x)
        quad = combinations[i, j]
        Z[k] = np.concatenate((x, quad), axis=None)

    return Z
```

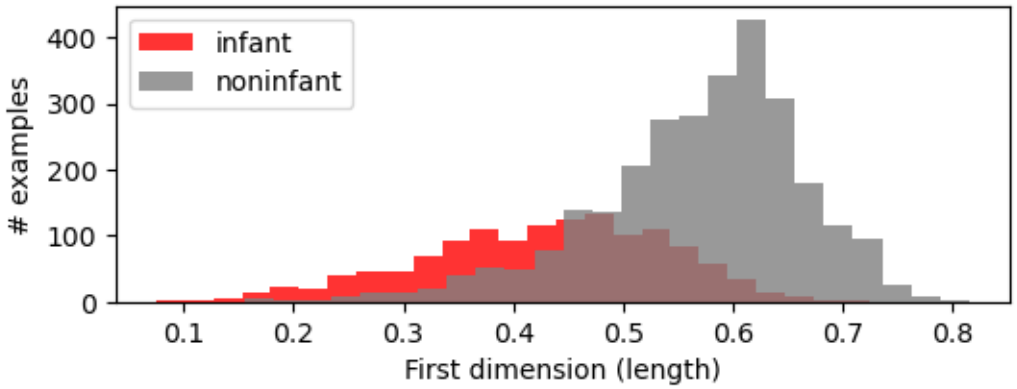
1.2 Analysis (5 + 5 = 10 P)

- **Print value of the objective function and the histogram for several values of `w`:**
  - `w` is a canonical coordinate vector for the first feature (length).
  - `w` is the difference between the mean vectors of the two classes.
  - `w` is the Fisher linear discriminant.
  - `w` is the Fisher linear discriminant (after quadratic expansion of the data).

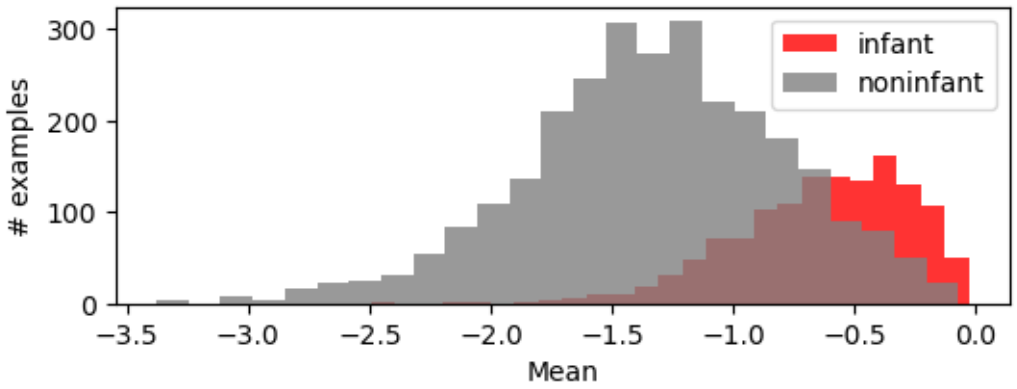
[13]:
```
X1 = abalone.I
X2 = abalone.N

w1 = np.array([1,0,0,0,0,0,0])
J1 = objective(X1, X2, w1)
print(f"First dimension (length): {J1:.5f}")
abalone.plot(w1, 'First dimension (length)')

w2 = np.mean(X1, axis=0) - np.mean(X2, axis=0)
w2 /= np.linalg.norm(w2)
J2 = objective(X1, X2, w2)
print(f"Means Linear: {J2:.5f}")
abalone.plot(w2, 'Mean')

w3 = fisher(X1, X2)
J3 = objective(X1, X2, w3)
```

```
print(f"Fisher: {J3:.5f}")
abalone.plot(w3, 'Fisher')

Z1 = expand(X1)
Z2 = expand(X2)
w4 = fisher(Z1, Z2)
J4 = objective(Z1, Z2, w4)
print(f"Fisher after expansion: {J4:.5f}")

plt.figure(figsize=(6,2))
plt.xlabel('Fisher after expansion')
plt.ylabel('# examples')
plt.hist(Z1 @ w4, bins=25, alpha=0.8,color='red',label='infant')
plt.hist(Z2 @ w4, bins=25, alpha=0.8,color='gray',label='noninfant')
plt.legend()
plt.show()
```

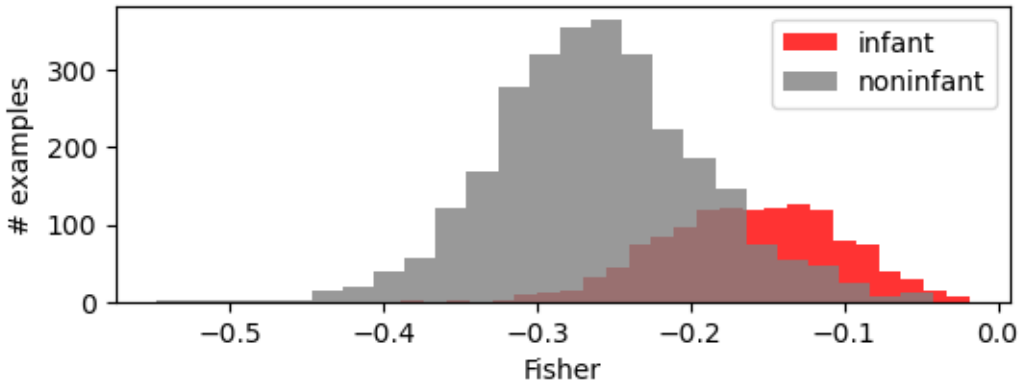
```
First dimension (length): 0.00048
```



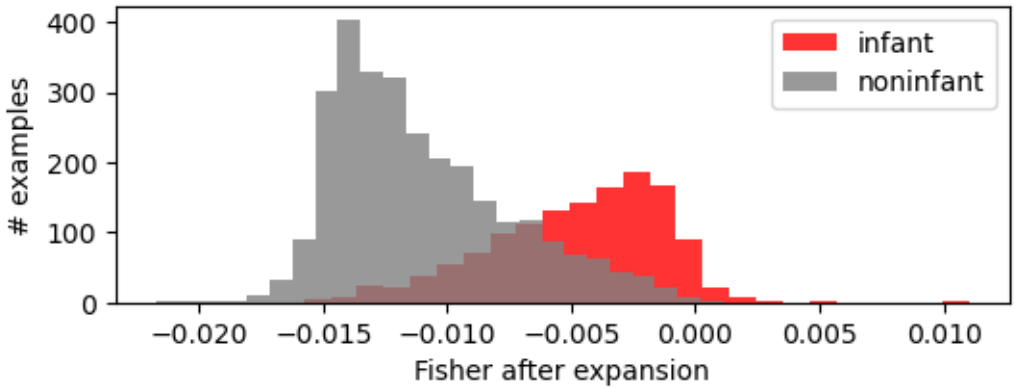

```
Means Linear: 0.00050
```



```
Fisher: 0.00057
```




```
Fisher after expansion: 0.00077
```




```
[ ]:
```